

PROGRAMMAZIONE DI RETE IN JAVA



DEFINIZIONE SOCKET TCP



Nel linguaggio comune
SOCKET = PRESA DI CORRENTE

Un socket può essere pensato come un canale tramite il quale i processi che sono messi in comunicazione possono inviare e ricevere dati. Si tratta quindi di un canale bidirezionale.

Porte



I protocolli coinvolti nell'implementazione dei Socket possono essere:

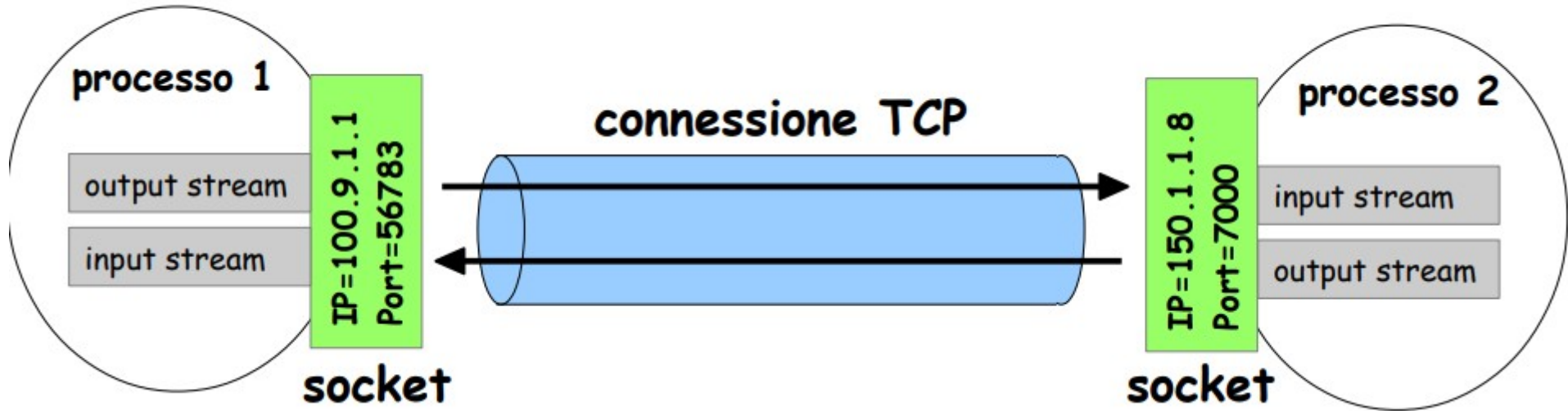
TCP (Transfer Control Protocol)

UDP (User Datagram Protocol)

Un socket TCP è un estremo (end-point) di una connessione TCP tra due processi.

Un socket è caratterizzato da indirizzo IP e numero di porta.

CONNESSIONE TCP



Una connessione TCP è un canale di comunicazione bidirezionale che consente a due processi di scambiare dati in modo affidabile tramite la rete.

Ognuno dei due estremi può leggere e scrivere dati sul canale.

Una connessione TCP è identificata da **4 valori**:

IP/porta del primo socket e IP/porta del secondo socket.

Le porte



I numeri di porta sono numeri a 16 bit il cui valore può variare tra **0 e 65535**.

Non tutti questi valori possono essere utilizzati.

- I numeri di porta compresi tra **0 e 1023 sono riservati** a particolari servizi (telnet, ftp, SMTP, POP3 e HTTP e possono essere utilizzati soltanto in caso di interfacciamento con tali servizi)
- Le porte utilizzabili dagli utenti della rete sono quelle definite **Registered Ports**, i cui valori variano tra **1024 e 49151**.
- Sui numeri di porta compresi tra **49152 e 65535** non viene applicato alcun controllo (sono definite **private ports**).

Sia il client sia il server devono stabilire la porta da utilizzare per la comunicazione, altrimenti non riusciranno a comunicare (anche se l'indirizzo IP è corretto).

Socket in Java



Il package Java che fornisce supporto per l'implementazione dei Socket è **java.net**.

Le classi **Socket** e **ServerSocket** sono le due classi necessarie per l'implementazione di una comunicazione client-server basata sui socket. La prima mentre la seconda .

Socket = si occupa della gestione del client che effettua la chiamata

ServerSocket = fornisce le funzionalità necessarie a creare un server che stia in ascolto su una particolare porta

Ip del dispositivo: `InetAddress.getLocalHost();`

Le classi in Java



Per stabilire una connessione sono necessari i seguenti passaggi:

- **Il server** sceglie una **porta** su cui mettersi in ascolto. Quando il client tenterà di aprire una connessione, il server richiamerà il **metodo accept()** della **classe ServerSocket** ottenendo un riferimento al canale di comunicazione instaurato con il client.
- **Il client** apre una connessione con il server con un oggetto di tipo **Socket** a cui verranno passati due parametri: **l'indirizzo IP del server e la porta su cui il server stesso è in ascolto**.

Una volta instaurata la connessione, server e client possono comunicare tramite gli stream (classi **InputStream** e **OutputStream**).

ESEMPIO



Vogliamo realizzare una semplice comunicazione tramite **TCP**:

- Il server ha indirizzo **IP 192.168.0.10** ed è in attesa di ricevere connessioni TCP sulla **porta 7000**
- il client instaura una connessione TCP con il server e rimane in attesa di ricevere dei dati
- il server, appena la connessione è instaurata, invia al client una stringa, poi chiude la connessione e termina.
- il client mostra a video la stringa ricevuta e termina.